for the Actuator endpoints, while all the others will keep using OpenID Connect and OAuth2. For simplicity, in the Polar Bookshop system, we'll keep the Actuator endpoints unauthenticated from inside the Kubernetes cluster and block any access to them from the outside (as you'll see in chapter 15).

WARNING In a real production scenario, I would recommend protecting access to the Actuator endpoints even from within the cluster.

Go to the `SecurityConfig` class of your Catalog Service project and update the Spring Security configuration to allow unauthenticated access to the Spring Boot Actuator endpoints.

Listing 13.6 Allowing unauthenticated access to the Actuator endpoints

```
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
     .authorizeHttpRequests(authorize -> authorize
       .mvcMatchers("/actuator/**").permitAll()                    ❶
       .mvcMatchers(HttpMethod.GET, "/", "/books/**").permitAll()
       .anyRequest().hasRole("employee")
     )
     .oauth2ResourceServer(OAuth2ResourceServerConfigurer::jwt)
     .sessionManagement(sessionManagement -> sessionManagement
       .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
     .csrf(AbstractHttpConfigurer::disable)
     .build();
  }
}
```

❶ Allows unauthenticated access to any Spring Boot Actuator endpoint

Finally, open the application.yml file in your Catalog Service project (catalog-service), and configure Actuator to expose the health HTTP endpoint. If you followed the examples in chapter 4, you might have an